ARCHITECTURE & PERFORMANCE GUIDE

Concept Performance Dashboard

Refresh & Cache Optimization

A Comprehensive Developer Guide explaining the Dashboard Architecture, Cache-Busting

Mechanics, and Redundant Backend Query Optimizations.

Prepared For Junior Developer Handover

Prepared By Antigravity Coding Assistant

Date & Version May 27, 2026 • v1.0

Executive Summary

The **Concept Performance** page (/ad-performance/concepts-v2) displays aggregated marketing data across multiple advertising networks (Facebook, TikTok, YouTube, AppLovin, and Google Ads). Historically, the page had a "Live Data" tab that was slow to query. Although the tab was removed from the UI, the "Refresh Cache" button still executed the slow, redundant background cache-warming routines associated with it, causing page refresh times to exceed 60 seconds.

This guide details the caching layers, the source of the performance bottlenecks, and the code modifications implemented to bring dashboard refresh times down to **1–10 seconds**.

1. What the "Refresh Cache" Button Does

When a user clicks the "Refresh Cache" button on the UI, the page sets an isRefreshing state to show a loading spinner and triggers the handleRefresh function.

The Optimized 5-Step Process

1. **UI State Shift:** The screen displays a "Refreshing..." status to block double clicks.

2. **Browser Cache Invalidation:** The client-side browser cache mapping (which stores loaded queries) is completely cleared.
3. **Request to Cached API:** The frontend makes a network call to the main dashboard data API: `/api/ad-performance/cached`.
4. **Hybrid Server Fetch:**
    - The server fetches historical pre-processed metrics from **PostgreSQL** (up to 2 days ago).
    - If the selected date range includes yesterday or today, it queries **BigQuery** for only those 2 "gap" days to capture real-time ad performance.
5. **Merge and Render:** The server combines both data sources and returns the JSON payload. The page renders the fresh data and displays a green checkmark indicating a successful refresh.

2. Cache Architecture: Browser vs. Server

To maintain a fast interface while handling millions of raw rows, the application employs caching at two distinct layers:

Aspect	Browser-Level Cache (Client)	In-Memory Cache (Server-Side)
Where it lives	Inside the individual user's browser RAM.	Inside the Next.js cloud server's RAM.
Who can see it	Only the specific user on that browser tab.	All users accessing the website.
Purpose	Makes UI pagination, sorting, and filter toggling instant.	Prevents hitting BigQuery on every page load, reducing GCP bills.

Aspect	Browser-Level Cache (Client)	In-Memory Cache (Server-Side)
Eviction / Expiry	LRU (max 50 entries) or 1-hour expiration.	1-hour fixed duration or manual bust.

The Browser Cache Analogy

Think of the **Browser Cache** as writing the daily menu on a sticky note on your own desk. It helps you lookup prices instantly. Think of the **Server Cache** as printing the menu on a bulletin board in the middle of the office. Everyone shares it, saving everyone from asking the kitchen separately.

3. The Legacy Bottlenecks & Why it was Slow

Previously, when a user clicked "Refresh Cache," the system executed three API requests in parallel. Two of these requests were legacy operations left over from the deleted "Live Data" tab:

1. **Legacy Call 1 (BigQuery Cache Warming):** `/api/ad-performance?cache=refresh`. This triggered a massive SQL scan on BigQuery over 14 tables dating back to **January 1, 2025**. This call took **15–60+ seconds**.
2. **Legacy Call 2 (Prisma Cache Warming):** `/api/ad-performance/combinations?cache=refresh`. This query pulled up to 100,000 mapping rows from PostgreSQL and updated a cache that the main page data API never even referenced. This took **1–5 seconds**.

Because the frontend blocked user interaction until all parallel requests resolved, users were forced to wait up to a minute for data loads that were never displayed on their screen.

4. Applied Optimizations

We modified `handleRefresh` in the main page component to eliminate these parallel calls. Since the server-side caches automatically refresh when they expire (every hour) and are maintained by cron jobs, it is safe to bypass them during a user-initiated table refresh.

Code Diff implemented in `page.tsx`:

```
// File: src/app/ad-performance/concepts-v2/page.tsx

    try {
-     console.log('🔄 Refreshing ad-performance + combinations caches...');
-     const [bigQueryResponse, combinationsResponse] = await Promise.all([
-       fetch('/api/ad-performance?cache=refresh', {
-         headers: { 'Cache-Control': 'no-cache' },
-       }),
-       fetch('/api/ad-performance/combinations?cache=refresh', {
-         headers: { 'Cache-Control': 'no-cache' },
-       }),
-     ]);
-
-     if (!bigQueryResponse.ok || !combinationsResponse.ok) {
-       console.warn('⚠️ Cache refresh failed — not refetching cached data', {
-         adPerformance: bigQueryResponse.status,
-         combinations: combinationsResponse.status,
-       });
-       flashFailed();
-       return;
-     }
+     console.log('🔄 Refreshing cache (clearing browser cache and refetching data)...');

      const { fromDate, toDate } = getDateRangeForAPI();
      const sortBy = cachedCurrentSort?.sortBy || 'totalSpend';
```

Optimization Performance Results

- **Past Ranges (e.g. Last Month):** Reduced from **60+ seconds** to **1–2 seconds** (queries PostgreSQL directly, completely skips BigQuery).
- **Recent Ranges (e.g. Last 7 Days):** Reduced from **60+ seconds** to **~10 seconds** (queries BigQuery only for the 2-day gap, instead of scanning everything since Jan 2025).

5. API Endpoint Directory

The following file paths define the ad performance API system:

/api/ad-performance

- **Location:** src/app/api/ad-performance/route.ts
- **Underlying Service:** src/lib/services/adPerformanceService.ts
- **Description:** Queries BigQuery since Jan 1, 2025. Keeps a 1-hour server-side cache. Used by AI suggestions, winning hook analytics, and cron jobs.

/api/ad-performance/combinations

- **Location:** src/app/api/ad-performance/combinations/route.ts
- **Underlying Service:** src/lib/services/combinationsService.ts
- **Description:** Loads mappings (Ad ID to concept) from PostgreSQL. Used by suggestions and other configuration pages.

/api/ad-performance/cached

- **Location:** src/app/api/ad-performance/cached/route.ts
- **Description:** The core API backing the Concepts-V2 dashboard. Reads historical

data from PostgreSQL and queries BigQuery specifically for the yesterday/today live data gap.

/api/ad-performance/filtered

- **Location:** src/app/api/ad-performance/filtered/route.ts
- **Description:** A helper API that queries BigQuery with strict date boundaries. Used by the cached endpoint to fetch the live gap metrics.